

Floor Map Builder

Visual room layout, availability state & drag-and-drop setup

Feature ID: FMB-001

Version: 1.0.0

Date: May 24, 2026

Status: Ready for development

Phase: Phase 1 — MVP Core

Priority: P0 — blocks all other room-level features

Depends on: Property setup · Room types · Floor config

Blocks: Booking creation · Availability calendar · Inventory view

Owner: Juan Carlos Del Agua Pascual (TEREN)

Stack: SvelteKit + Tailwind v4 · Go API · PostgreSQL

This document is the single source of truth for building the Floor Map Builder feature. It covers UX flows, component breakdown, API contracts, data model, business rules, acceptance criteria, and open questions. A developer should be able to start from scratch using only this document.

1. Overview & Purpose

The Floor Map Builder is the visual command center of TEREN Hotels. It serves two distinct modes: **Setup mode** (owner configures the property layout once) and **Operations mode** (receptionist sees live room status and assigns rooms to bookings). It is the first screen staff open every morning and the last they check before end of shift.

Mode	User	Primary action	Frequency
Setup	Owner	Add floors, place rooms, assign types	Once at onboarding, rarely after
Operations	Receptionist	View room status, assign room to booking	Every shift
Operations	Owner	Overview of occupancy + revenue snapshot	Daily

1.1 Core principles for this feature

- Zero training required — layout must be self-evident
- Status of every room visible in 1 second
- Room assignment takes 2 taps from the map
- No modal forms for common actions (inline editing)
- Works on a 10" tablet at the reception desk
- Outdoor-readable contrast (TEREN brand rule)
- Smooth status transitions — no hard flickers
- pos_x / pos_y stored in DB — no library lock-in
- Floor tabs for multi-floor navigation
- Graceful on < 10 rooms and up to 50 rooms

2. UX Flows

2.1 Setup flow — owner configures the property layout

This flow runs once during onboarding. The owner defines floors, places rooms on a grid, assigns each room a type, and names them. After saving, the map is immediately visible in Operations mode.

Step	Screen / Component	Action	Result
1	Property Settings → Floors	Owner clicks '+ Add floor', enters label (e.g. 'Ground', 'Floor 1')	Floor tab created, sort_order assigned
2	Floor Map — Setup mode	Floor tab is selected, empty grid is shown (12×8 cells default)	Blank canvas ready
3	Room palette (sidebar)	Owner drags a room token from palette onto a grid cell	Room placed at pos_x, pos_y — default name auto-assigned (e.g. '101')
4	Room token (inline)	Owner clicks room → inline editor appears: name, room type selector	PATCH /rooms/{id} on blur
5	Room token (drag)	Owner drags placed room to another cell	pos_x / pos_y updated on drop
6	Room token (delete)	Owner right-clicks or long-presses → 'Remove room'	Room soft-deleted if no bookings, hard warning if bookings exist
7	Save bar (sticky bottom)	Owner clicks 'Save layout'	Batch PATCH /rooms with updated positions

2.2 Operations flow — receptionist assigns a room to a booking

This is the highest-frequency interaction. A guest arrives, receptionist opens the map, sees available rooms highlighted, taps one, assigns the booking. Done in under 10 seconds.

Step	Screen / Component	Action	Result
1	Dashboard → Floor Map button	Receptionist opens floor map (default: today's date)	Map loads with real-time status for today
2	Date picker (top bar)	Optionally change date or select range for a pending booking	Room status recalculates for selected range
3	Room token (green = available)	Receptionist taps an available room	Room detail drawer slides in from right
4	Room detail drawer	Drawer shows: room name, type, rate for selected dates, current/next booking	Assign button visible if room is available
5	Assign button in drawer	Receptionist taps 'Assign to booking' → selects pending booking from list	PATCH /bookings/{id} with room_id
6	Floor map (live update)	Room token transitions to 'pending' state (amber)	No page reload — reactive update

7	Check-in action	From drawer: 'Check in' button → confirms arrival	Booking status → checked_in, room → occupied (red)
---	-----------------	---	--

2.3 Block flow — owner/receptionist blocks a room manually

Step	Action	Result
1	Tap any non-occupied room → Room detail drawer opens	Drawer shows current status
2	Tap 'Block room' → select reason (maintenance / owner use / out of service) + date range + optional note	POST /room-blocks
3	Room transitions to blocked state (striped pattern, dark bg)	Blocked rooms excluded from availability queries
4	To unblock: tap room → 'Remove block' in drawer	DELETE /room-blocks/{id}

3. Room Status States

Each room token on the map renders one of five states. State is computed at query time — never stored as a derived field to avoid sync issues.

State	Color	Condition	User action available
available	Green #16A34A	No booking or block overlapping selected date range	Assign booking · Block room
occupied	Red #DC2626	Booking with status checked_in overlaps date range	View guest · Check out
pending	Amber #D97706	Booking confirmed but not yet checked in (future or today)	Check in · View booking · Reassign
blocked	Dark striped #44403C	ROOM_BLOCK record overlaps date range	View block reason · Remove block
inactive	Gray #A8A29E	ROOM.status = inactive (not available for rent)	Activate room (owner only)

TEREN brand rule: status colors must pass WCAG AA contrast against the Warm Stone background (#F5F4F1). Green, red, and amber all do. The blocked striped pattern uses a CSS repeating-linear-gradient so no image assets are needed.

3.1 Availability computation (backend)

The API computes availability by querying against BOOKING and ROOM_BLOCK simultaneously. The frontend never computes state — it only renders what the API returns.

```
-- Availability query for a property on a given date range
SELECT
  r.id,
  r.number,
  r.room_type_id,
  r.pos_x,
  r.pos_y,
  r.status AS room_status,
  CASE
    WHEN r.status = 'inactive' THEN 'inactive'
    WHEN rb.id IS NOT NULL THEN 'blocked'
    WHEN b_in.id IS NOT NULL THEN 'occupied'
    WHEN b_conf.id IS NOT NULL THEN 'pending'
    ELSE 'available'
  END AS availability_state,
  b_in.id AS active_booking_id,
  b_conf.id AS pending_booking_id,
  rb.id AS block_id
FROM rooms r
LEFT JOIN room_blocks rb
ON rb.room_id = r.id
AND rb.start_date < $end_date AND rb.end_date > $start_date
```

```
LEFT JOIN bookings b_in
ON b_in.room_id = r.id
AND b_in.status = 'checked_in'
AND b_in.check_in < $end_date AND b_in.check_out > $start_date
LEFT JOIN bookings b_conf
ON b_conf.room_id = r.id
AND b_conf.status = 'confirmed'
AND b_conf.check_in < $end_date AND b_conf.check_out > $start_date
WHERE r.property_id = $property_id
ORDER BY r.floor_id, r.pos_y, r.pos_x;
```

4. Component Breakdown (SvelteKit)

Components follow atomic design. All are Svelte 5 runes-based (if available) or Svelte 4 reactive stores. No external drag-and-drop library — native HTML5 drag-and-drop API for setup mode.

Component	File path	Responsibility	Props / Events
FloorMap	routes/map/+page.svelte	Page container, mode switcher, date picker	mode: 'setup' 'ops'
FloorTabs	lib/components/map/FloorTabs.svelte	Floor navigation tabs, add floor button (setup)	floors[], activeFloorId
RoomGrid	lib/components/map/RoomGrid.svelte	12x8 CSS grid, renders RoomToken for each room, drop zone in setup	rooms[], mode, onDrop
RoomToken	lib/components/map/RoomToken.svelte	Individual room cell — color, label, drag handle, status icon	room, state, onClick
RoomPalette	lib/components/map/RoomPalette.svelte	Sidebar in setup mode: draggable new room tokens per type	roomTypes[], onDragStart
RoomDrawer	lib/components/map/RoomDrawer.svelte	Slide-in panel from right: room details, assign/block/check-in	room, bookings, onAction
StatusLegend	lib/components/map/StatusLegend.svelte	Color legend bar below the grid	—
BlockModal	lib/components/map/BlockModal.svelte	Modal for block reason + date range + note	roomId, onConfirm
DateRangePicker	lib/components/ui/DateRangePicker.svelte	Compact date range selector in ops mode header	value, onChange
OccupancyBar	lib/components/map/OccupancyBar.svelte	Top bar showing X/Y rooms occupied today + RevPAR snapshot	stats

4.1 Grid system

The room grid is a CSS grid with 12 columns and dynamic rows. Each cell is 56×56px on desktop, 48×48px on tablet. Rooms occupy exactly one cell. Gap between cells: 6px. Grid does not scroll horizontally — if a property needs more than 12 rooms per row, they should split into floors. This constraint simplifies the drag-and-drop logic significantly.

```
/* RoomGrid.svelte — grid container */
.room-grid {
display: grid;
grid-template-columns: repeat(12, 56px);
grid-auto-rows: 56px;
gap: 6px;
padding: 16px;
background: var(--color-bg-warm); /* #F5F4F1 */
border-radius: 12px;
}

/* Each room token placed via CSS grid-column / grid-row */
.room-token {
grid-column: calc(var(--pos-x) + 1); /* pos_x from DB */
```

```
grid-row: calc(var(--pos-y) + 1); /* pos_y from DB */  
}
```


5. API Contract (Go backend)

All endpoints are under `/api/v1`. Auth via Bearer JWT. Property scoped — all queries implicitly filter by `property_id` extracted from the JWT claims.

Method	Endpoint	Description	Auth
GET	<code>/floors</code>	List floors for property (sorted by <code>sort_order</code>)	owner · receptionist
POST	<code>/floors</code>	Create floor { <code>floor_number</code> , <code>label</code> , <code>sort_order</code> }	owner
PATCH	<code>/floors/{id}</code>	Update floor label or <code>sort_order</code>	owner
DELETE	<code>/floors/{id}</code>	Delete floor — only if no rooms attached	owner
GET	<code>/rooms</code>	List all rooms with type info	owner · receptionist
POST	<code>/rooms</code>	Create room { <code>floor_id</code> , <code>room_type_id</code> , <code>number</code> , <code>pos_x</code> , <code>pos_y</code> }	owner
PATCH	<code>/rooms/{id}</code>	Update room (<code>number</code> , <code>type</code> , <code>pos_x</code> , <code>pos_y</code> , <code>status</code>)	owner
DELETE	<code>/rooms/{id}</code>	Soft-delete room (inactive) if has bookings; hard-delete if clean	owner
PATCH	<code>/rooms/positions</code>	Batch update positions [{ <code>id</code> , <code>pos_x</code> , <code>pos_y</code> }] — called on layout save	owner
GET	<code>/map?date_from=&date;_to=</code>	Floor map with availability state per room for date range	owner · receptionist
POST	<code>/room-blocks</code>	Create block { <code>room_id</code> , <code>start_date</code> , <code>end_date</code> , <code>reason</code> , <code>notes</code> }	owner · receptionist
DELETE	<code>/room-blocks/{id}</code>	Remove block	owner · receptionist

5.1 GET /map — response shape

```
// GET /api/v1/map?date_from=2025-06-01&date;_to=2025-06-02
```

```
{
  "property_id": "uuid",
  "date_from": "2025-06-01",
  "date_to": "2025-06-02",
  "floors": [
    {
      "id": "uuid",
      "label": "Ground Floor",
      "floor_number": 0,
      "sort_order": 0,
      "rooms": [
        {
          "id": "uuid",
```

```
"number": "101",
"pos_x": 0,
"pos_y": 0,
"room_type": { "id": "uuid", "name": "Standard" },
"availability": "available",
"active_booking": null,
"pending_booking": null,
"block": null
}
]
}
]
}
```

5.2 PATCH /rooms/positions — batch update

```
// Called when owner clicks 'Save layout' in Setup mode
// PATCH /api/v1/rooms/positions
{
  "positions": [
    { "id": "uuid-101", "pos_x": 0, "pos_y": 0 },
    { "id": "uuid-102", "pos_x": 1, "pos_y": 0 },
    { "id": "uuid-103", "pos_x": 2, "pos_y": 0 }
  ]
}

// Response: 200 OK { updated: 3 }
// Use a single DB transaction – all-or-nothing update
```

6. Data Model — SQL Migrations

Two tables are new in this feature: FLOOR and ROOM_BLOCK. ROOM already exists from the base schema but gains pos_x / pos_y. All migrations use golang-migrate. File naming: {version}_{description}.up.sql

Migration 001 — floors table

```
-- 001_create_floors.up.sql

CREATE TABLE floors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id UUID NOT NULL REFERENCES properties(id) ON DELETE CASCADE,
  floor_number INT NOT NULL,
  label TEXT NOT NULL,
  sort_order INT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (property_id, floor_number)
);

CREATE INDEX idx_floors_property ON floors(property_id);
```

Migration 002 — rooms table with positions

```
-- 002_create_rooms.up.sql

CREATE TYPE room_status AS ENUM ('active', 'inactive');

CREATE TABLE rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id UUID NOT NULL REFERENCES properties(id) ON DELETE CASCADE,
  floor_id UUID NOT NULL REFERENCES floors(id) ON DELETE RESTRICT,
  room_type_id UUID NOT NULL REFERENCES room_types(id) ON DELETE RESTRICT,
  number TEXT NOT NULL,
  status room_status NOT NULL DEFAULT 'active',
  pos_x INT NOT NULL DEFAULT 0,
  pos_y INT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (property_id, number),
  UNIQUE (floor_id, pos_x, pos_y) -- no two rooms on same cell
);

CREATE INDEX idx_rooms_property ON rooms(property_id);
CREATE INDEX idx_rooms_floor ON rooms(floor_id);
```

Migration 003 — room_blocks table

```
-- 003_create_room_blocks.up.sql

CREATE TYPE block_reason AS ENUM ('maintenance', 'owner_use', 'out_of_service');

CREATE TABLE room_blocks (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
created_by UUID NOT NULL REFERENCES users(id),
start_date DATE NOT NULL,
end_date DATE NOT NULL,
reason block_reason NOT NULL,
notes TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CHECK (end_date > start_date)
);
CREATE INDEX idx_room_blocks_room ON room_blocks(room_id);
CREATE INDEX idx_room_blocks_dates ON room_blocks(start_date, end_date);
```

7. Business Rules & Validation

#	Rule	Where enforced	Severity
BR-01	Two rooms cannot share the same (floor_id, pos_x, pos_y). DB unique constraint + API validation before insert.	DB + API	Hard block
BR-02	A room cannot be deleted if it has any booking (any status except cancelled). API returns 409 with list of conflicting bookings.	API	Hard block
BR-03	A room cannot be blocked if it already has an active checked_in booking overlapping the date range. API returns 409.	API	Hard block
BR-04	A booking cannot be assigned to a room that is blocked or occupied for that date range. API returns 409.	API	Hard block
BR-05	Minimum floor requirement: a property must have at least 1 floor before rooms can be created.	API + UI	Hard block
BR-06	pos_x must be 0–11, pos_y must be 0–19 (12 col × 20 row max grid). Values outside range rejected.	API	Hard block
BR-07	Floor deletion: only allowed if floor has zero rooms. API returns 409 otherwise.	API	Hard block
BR-08	Room number must be unique per property (not per floor). Allows '101' on floor 1 and '201' on floor 2 but not two '101'.	DB + API	Hard block
BR-09	Changing a room's type is allowed at any time — does not invalidate existing bookings (rates are stored on booking, not room type).	—	Info
BR-10	Owner can override a block conflict warning and force-assign (not a hard block at application level — only a warning prompt).	UI only	Warning

7.1 Role permissions matrix

Action	Owner	Receptionist
View floor map (operations mode)	Yes	Yes
Add / remove floors	Yes	No
Place / move / delete rooms	Yes	No
Assign room type	Yes	No
Block a room	Yes	Yes
Remove a block	Yes	Yes
Assign room to booking	Yes	Yes
Check in / check out	Yes	Yes
View OccupancyBar + RevPAR	Yes	No (hidden)

8. Acceptance Criteria

All criteria below must pass before this feature is considered done. Each criterion maps to a testable scenario — unit, integration, or manual E2E.

ID	Criterion	Type
AC-01	Owner can create a floor, place 10 rooms on the grid, save — all positions persist after page reload.	E2E
AC-02	Dragging a room to an occupied cell shows a visual conflict indicator and reverts on drop.	E2E
AC-03	GET /map returns correct availability state for all 5 states (available, occupied, pending, blocked, inactive) given the right DB fixtures.	Integration
AC-04	Two rooms cannot be saved to the same (floor_id, pos_x, pos_y) — API returns 409.	Integration
AC-05	Receptionist taps available room → drawer opens → assigns to confirmed booking → token turns amber without page reload.	E2E
AC-06	Check-in action → booking status → checked_in, room token → red. Confirmed in DB and UI.	E2E
AC-07	Blocking a room with an active checked_in booking for overlapping dates returns 409.	Integration
AC-08	Deleting a room with existing non-cancelled bookings returns 409 with booking list.	Integration
AC-09	Floor map loads in under 1.5s on a 50-room property on a standard 4G connection (Singapore region server).	Performance
AC-10	All room status colors pass WCAG AA contrast against #F5F4F1 background.	Accessibility
AC-11	Receptionist role cannot see Setup mode tab or OccupancyBar RevPAR figure — UI hides and API returns 403.	Security
AC-12	pos_x outside 0–11 or pos_y outside 0–19 is rejected by API with 422.	Integration
AC-13	PATCH /rooms/positions with 50 rooms completes in a single DB transaction — if one fails, none are updated.	Integration
AC-14	Floor map is usable on a 1024×768 tablet viewport — no horizontal scroll, all room tokens remain tappable (min 44×44px touch target).	Manual / visual

9. UI States & Edge Cases

Scenario	Expected behavior
Property has 0 floors	Full-page empty state: 'Start by adding your first floor' + CTA button. No grid shown.
Floor has 0 rooms	Empty grid with dashed border + 'Drag a room here to get started' hint text (setup mode only).
All rooms occupied today	OccupancyBar shows 100% · All tokens red · No available rooms message in drawer if tapped.
Date picker set to past date	Map renders historical state (read-only). All actions (assign, block, check-in) are disabled. Banner: 'Viewing past state — read only.'
Room number collision on save	Inline error on the conflicting token: 'Room number already exists'. Save button disabled until resolved.
Network error on drag-and-drop save	Toast: 'Layout not saved — check connection.' Positions revert to last saved state.
50 rooms on one floor	Grid renders all rooms. Tokens scale down to 48×48px. Horizontal scroll allowed only on mobile (<768px).
Block with start_date = end_date	UI date picker enforces end_date > start_date. Submit button disabled otherwise.
Room type deleted after room creation	Room shows 'Type removed' label in orange. Does not break map. Owner prompted to reassign type.

10. Open Questions

#	Question	Impact	Status
OQ-01	Should the grid support rooms larger than 1×1 cell? (e.g. suites spanning 2 cells). Adds significant complexity to drag logic and DB model.	High — changes pos_w / pos_h fields in schema	Open
OQ-02	Should Setup mode allow uploading a floor plan image as a background layer? (helps owner place rooms accurately).	Medium — adds S3 upload + image overlay component	Open
OQ-03	Should the OccupancyBar be visible to receptionists in a limited form (no revenue, only occupancy count)?	Low — UI change only	Open
OQ-04	What is the max number of floors per property? Current assumption: 10. Confirm with user research.	Low — validation rule only	Assumed: 10
OQ-05	Should blocked rooms show the block reason on the token itself (tooltip / icon) or only in the drawer?	Low — UX preference	Open

Flow systems. Soulful experiences.

TEREN Hotels · Floor Map Builder Feature Spec v1.0 · May 2026 · hello@teren.dev